

Computer Hardware

A Brief History of Concurrency: From a Button to a Scheduler

Ksawery Moźdzynski

Agenda For Today

1. Interrupts

The device tells the processor that something happened

2. The scheduler

One processor, several tasks

3. FreeRTOS

A real scheduler on the ESP32-C3

Today you will need

Wokwi



wokwi.com

your project **Circuit** from Lecture 5

ConcurrencySimulator

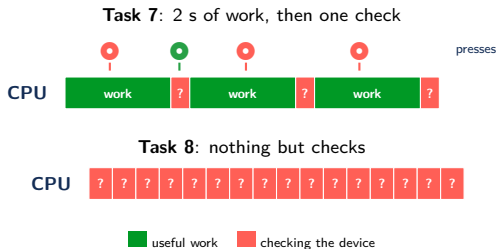


ksawerym.github.io/ConcurrencySimulator

new: made for today

Please open both now. All day we do the same thing: **a short talk, then you try it.**

From last time: polling



Check rarely (Task 7)

Short presses were **lost**: the LCD showed fewer than 5, often 0 or 1.

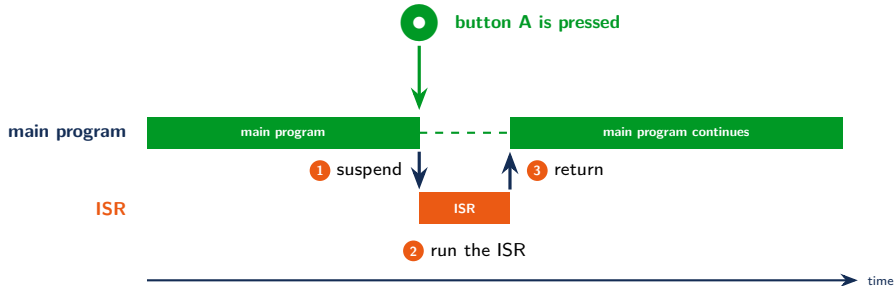
Check all the time (Task 8)

Nothing was lost, but more than **99.99%** of the checks found nothing.

Better idea: let the **button** tell the **processor** when it is pressed.

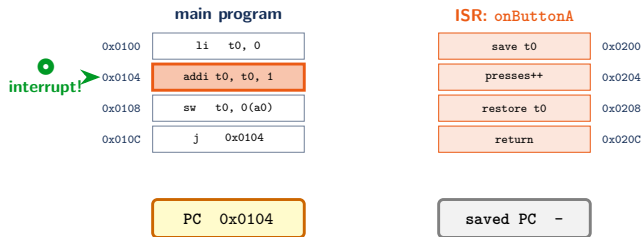
An interrupt

An **interrupt** is a signal from a device to the processor: *something happened, deal with it now*.



ISR = Interrupt Service Routine: a short function that the processor runs when the interrupt arrives.

What happens inside the processor

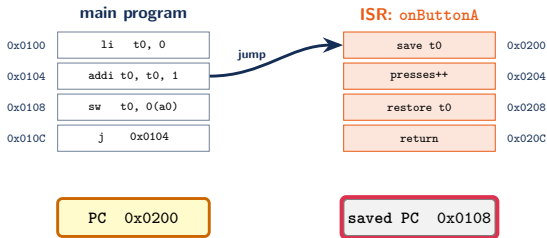


1 Signal

Button A raises an interrupt signal while the CPU runs addi.

In RISC-V the saved PC is kept in a special register, mepc.

What happens inside the processor



1 Signal

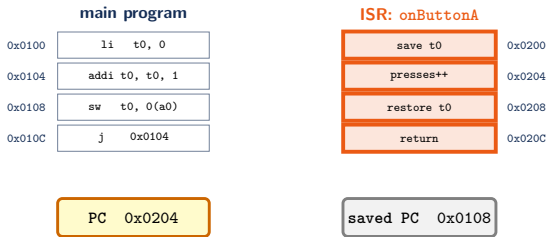
Button A raises an interrupt signal while the CPU runs addi.

2 Save and jump

The CPU **finishes** addi, **saves the PC** (the address of the next instruction) and jumps to the ISR.

In RISC-V the saved PC is kept in a special register, mepc.

What happens inside the processor



In RISC-V the saved PC is kept in a special register, `mepc`.

1 Signal

Button A raises an interrupt signal while the CPU runs `addi`.

2 Save and jump

The CPU **finishes** addi, **saves the PC** (the address of the next instruction) and jumps to the ISR.

3 The ISR runs

It saves t_0 , does its work and restores t_0 .

Simulator 1: polling and interrupts

Simulator task

Open **ConcurrencySimulator**, topic **Polling vs interrupt**.

1. Mode *2 s of work, then one check*. Click **Start**, then **5 short clicks**. How many presses does polling count? And the interrupt?
2. Look at the row *CPU: interrupts*. How much of the CPU time went to the ISR?

Then

Topic **Interrupt step by step**. The signal from button A arrives **during** the instruction at 0x0108.



```
0x0100  li    t0, 0
0x0104  addi  t0, t0, 1
0x0108  sw    t0, 0(a0)
0x010C  j     0x0104
```

3. **Predict** the saved PC. Then step through and check.
4. The ISR uses t0. What is t0 in the main program after the return?

Solution: Simulator 1

Question 1

Polling: usually **0 or 1**. A click counts only if it happens exactly during a check. Interrupt: **5**.

Question 2

Almost **nothing**. Each ISR takes a few microseconds: on the timeline it is only a thin orange line.

Question 3

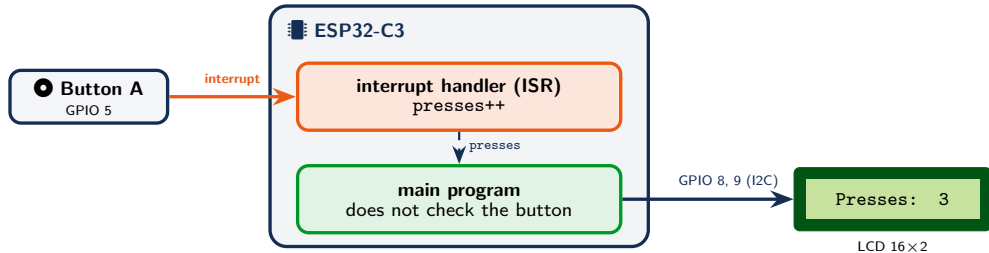
0x010C. The CPU first finishes `sw`, so the saved PC is the address of the **next** instruction.

Question 4

The same as before the interrupt. The ISR saved `t0` on the stack and restored it before the return.

An interrupt misses nothing, costs almost no CPU time, and the main program continues **exactly where it stopped**.

The button with an interrupt



The main program only **shows** the number. It never checks the button.

Interrupts in the code



```
const int BUTTON_A = 5;
volatile int presses = 0;

void IRAM_ATTR onButtonA() {    // the ISR
    presses++;
}

void setup() {
    pinMode(BUTTON_A, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(BUTTON_A),
                    onButtonA, FALLING);
}
```

attachInterrupt

Call onButtonA on a FALLING edge: when the pin goes from 1 to 0, i.e. when the button is **pressed**.

IRAM_ATTR

Keep the ISR in fast internal memory. Required on the ESP32.

volatile

As in Lecture 5: really read the variable every time. The ISR can change it “behind the program’s back”.

Task 1: count the presses

```
● ● ●  
#include <Wire.h>  
#include <LiquidCrystal_I2C.h>  
LiquidCrystal_I2C lcd(0x27, 16, 2);  
  
const int BUTTON_A = 5;  
volatile int presses = 0;  
// TODO: the ISR  
  
void setup() {  
  Wire.begin(8, 9);  
  lcd.init(); lcd.backlight();  
  // TODO: the pin and the interrupt  
}  
  
void loop() {  
  // TODO: show "Presses: N"  
}
```

Task

Open your project **Circuit** from Lecture 5: button A on GPIO 5, LCD on GPIO 8/9.

1. Count the presses of button A **with an interrupt**.
2. Show the number on the LCD: Presses: N.
3. Add `delay(2000)` to `loop()`, as in Task 7 of Lecture 5. Do you still lose any presses? Why not?

If you finish early: give button B (GPIO 6) its own interrupt that sets the counter back to 0.

Solution: count the presses

```
● ● ●
void IRAM_ATTR onButtonA() { presses++; }

void setup() {
  Wire.begin(8, 9);
  lcd.init(); lcd.backlight();
  pinMode(BUTTON_A, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(BUTTON_A),
                  onButtonA, FALLING);
}

void loop() {
  lcd.setCursor(0, 0);
  lcd.print("Presses: ");
  lcd.print(presses);
  lcd.print("    ");           // clear old digits
  delay(2000);
}
```

Question 3

No press is lost. The interrupt counts it even while loop() waits. The LCD just shows the new number up to 2 s later.

Compare with Task 7

The same delay(2000), but now nothing is missed.

If you finished early

onButtonB() sets presses = 0. Attach it to pin 6 with FALLING, like button A.

Rules for an ISR

1 Keep it short

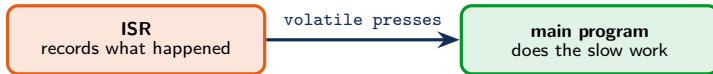
While the ISR runs, the main program waits.

2 No slow work inside

No `delay()`, no printing to the LCD or to Serial.

3 Share data through volatile variables

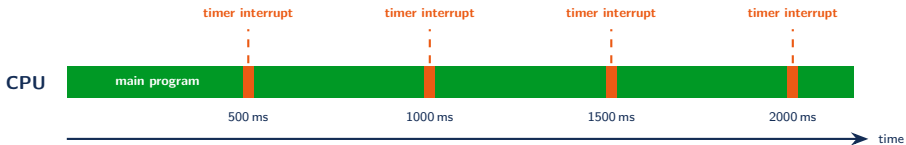
The ISR changes them, the main program reads them.



The ISR only **records** what happened. The main program **does the work**.

Interrupts from a timer

A **timer** is a counter inside the processor. When it reaches a set value, it raises an interrupt: again and again, at regular intervals.



A timer interrupt lets the processor do something **regularly**, without checking the time in the main program.

A timer in the code

```
● ● ●  
const int LED = 4;  
hw_timer_t *timer = nullptr;  
volatile bool ledOn = false;  
  
void IRAM_ATTR onTimer() {           // the ISR  
    ledOn = !ledOn;  
    digitalWrite(LED, ledOn);  
}  
  
void setup() {  
    pinMode(LED, OUTPUT);  
    timer = timerBegin(1000000);      // 1 step = 1 us  
    timerAttachInterrupt(timer, &onTimer);  
    timerAlarm(timer, 500000, true, 0); // every 500 ms  
}
```

`timerBegin(1000000)`

The timer counts at 1 MHz: one step every 1 μ s.

`timerAttachInterrupt`

When the alarm goes off, call `onTimer`.

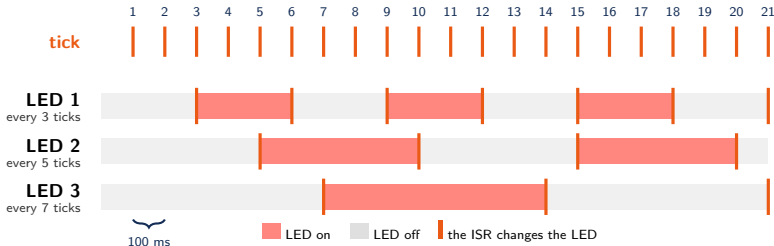
`timerAlarm`

Alarm after 500 000 steps = 500 ms, and **repeat** it (true).

The LED blinks with **no code in loop()**: the timer calls `onTimer` every 500 ms.

Three LEDs, three rhythms, one timer

Three LEDs should change every **300**, **500** and **700 ms**. But the ESP32-C3 has only **two** hardware timers.



One timer interrupt at a fixed rate is called a **tick**. Here: a tick every 100 ms. At every tick, the ISR **decides** what to do: LED 1 every 3 ticks, LED 2 every 5, LED 3 every 7.

The tick in the code



```
volatile unsigned long ticks = 0;
volatile bool led1 = false, led2 = false, led3 = false;

void IRAM_ATTR onTick() {           // every 100 ms
    ticks++;
    if (ticks % 3 == 0) { led1 = !led1; digitalWrite(LED1, led1); }
    if (ticks % 5 == 0) { led2 = !led2; digitalWrite(LED2, led2); }
    if (ticks % 7 == 0) { led3 = !led3; digitalWrite(LED3, led3); }
}
```

`ticks % 3 == 0`
Every third tick: 300 ms.

One timer
`timerAlarm(timer, 100000, true, 0):`
a tick every 100 ms.

Remember this idea: a **regular tick**, and a **decision** at every tick. We will need it in Part 2.

Simulator 2: the tick

Simulator task

Topic **Timer and tick**: a tick every 100 ms. LED 1 changes every 3 ticks, LED 2 every 5, LED 3 every 7. At the start all LEDs are off. **Predict first**, then check.

1. Which LEDs change at tick **15**? At tick **21**? At tick **35**?
2. Which LEDs are **on** right after tick **10**?
3. At which tick do all three change **together** for the first time? After how many seconds?

	LED 1	LED 2	LED 3
changes at tick 15	...	...	...
changes at tick 21	...	...	...
changes at tick 35	...	...	...
on after tick 10	...	...	...

Solution: Simulator 2

	LED 1	LED 2	LED 3
changes at tick 15	✓	✓	–
changes at tick 21	✓	–	✓
changes at tick 35	–	✓	✓
on after tick 10	on	off	on

Question 2

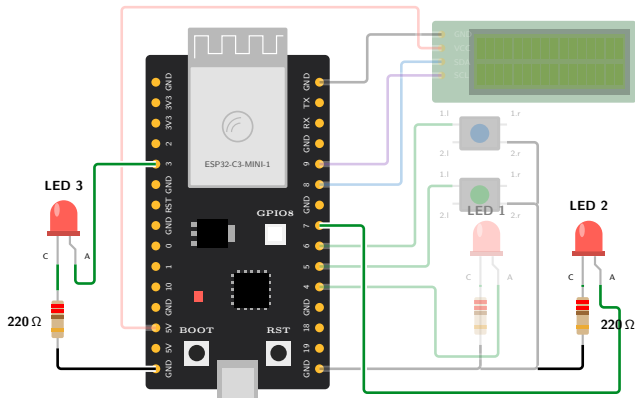
LED 1 changed 3 times (ticks 3, 6, 9): on. LED 2 twice (5, 10): off. LED 3 once (7): on.

Question 3

At tick $3 \times 5 \times 7 = 105$, after **10.5 s**.

A LED changes when the tick number is divisible by its period: the ISR checks this at every tick.

Build it: two more LEDs



1 Parts

Click +, add two **LEDs** and two **Resistors** (220 Ω).

2 Wires

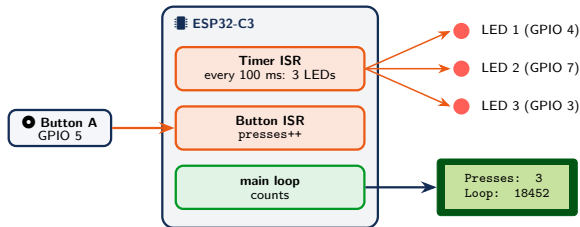
GPIO 7 $\rightarrow$ LED 2 A. GPIO 3 $\rightarrow$ LED 3 A. Each C $\rightarrow$ resistor $\rightarrow$ **GND**.

3 Keep it

The LED on GPIO 4 is now **LED 1**. Save the project.

Our circuit now: LEDs on GPIO 4, 7 and 3; buttons on 5 and 6; LCD on 8 and 9.

Task 2: three LEDs



Task

Use the circuit with three LEDs (or the project **Three LEDs** from the course page).

1. Make LED 1 blink every 500 ms **with a timer interrupt**, as on the slide “A timer in the code”.
2. Change it: one tick every **100 ms**, and the three LEDs change every 300, 500 and 700 ms.
3. Keep the button counter from Task 1. In `loop()`, also count as fast as possible and show Loop: N in row 1. Do the LEDs keep their rhythm?

If you finish early: make LED 1 blink with `millis()` in `loop()` instead of the timer, and keep the LCD writes. Is its rhythm still even?

Solution: three LEDs



```
const int LED1 = 4, LED2 = 7, LED3 = 3;
long loops = 0;
void setup() {
    // the LCD and button A as in Task 1
    pinMode(LED1, OUTPUT);  pinMode(LED2, OUTPUT);
    pinMode(LED3, OUTPUT);
    timer = timerBegin(1000000);
    timerAttachInterrupt(timer, &onTick);
    timerAlarm(timer, 100000, true, 0); // 100 ms
}

void loop() {
    loops++;
    lcd.setCursor(0, 0);
    lcd.print("Presses: ");  lcd.print(presses);
    lcd.setCursor(0, 1);
    lcd.print("Loop: ");      lcd.print(loops);
}
```

The ISR

onTick from the slide “The tick in the code”.

Question 3

Yes. The LEDs and the button are handled by interrupts, so it does not matter what loop() does, even slow LCD writes.

Simulator 3: could we do this without interrupts?

With delay()

```

● ● ●
void loop() {
    digitalWrite(LED1, HIGH); delay(300);
    digitalWrite(LED1, LOW);  delay(300);
    // and LED 2? LED 3?
}

```

With millis()

```

● ● ●
void loop() {
    if (millis() - last1 >= 300) {
        last1 += 300; toggle(LED1);
    }
    // the same for LED 2 and LED 3
    showOnLcd();           // several ms
}

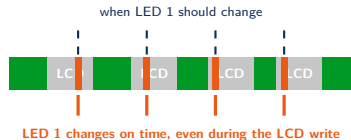
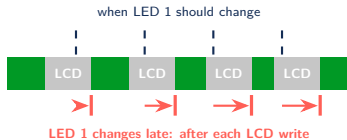
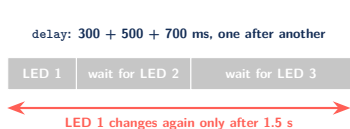
```

Simulator task

Topic **Timer and tick**, switch *How*: delay(), millis() in loop() or a timer interrupt. Turn on *loop()* writes to the LCD.

1. delay(): do the LEDs keep 300, 500 and 700 ms?
2. millis() in loop(): look at the moments when LED 1 changes. Are they on time?
3. Timer interrupt: are they on time now?

Solution: Simulator 3



1. delay()

No. While the CPU waits for one LED, it cannot serve the others. The waits add up.

2. millis()

Almost. But each slow step in the loop delays the LEDs: the rhythm is **uneven**.

3. Timer interrupt

Yes. The tick comes on time, whatever the loop is doing.

`millis()` in `loop()` is polling again: the loop keeps asking “is it time yet?”.

Without interrupts it is possible, but it is **hard to keep an exact rhythm** while the program also does other work.

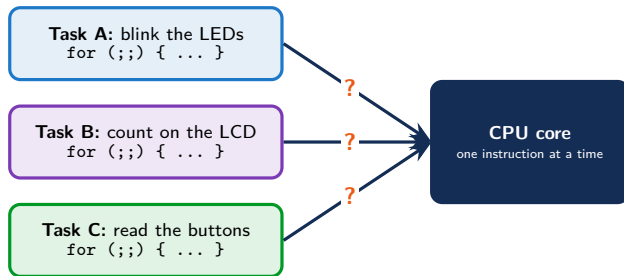
Interrupts: summary

Polling	Interrupts
the processor asks the device	the device tells the processor
short events can be missed	nothing is missed
wastes time on checking	the processor works until something happens
a rhythm is hard to keep	a timer gives an exact rhythm

Our tick decided **which LED** to change. Next: what if the tick decided **which program** runs?

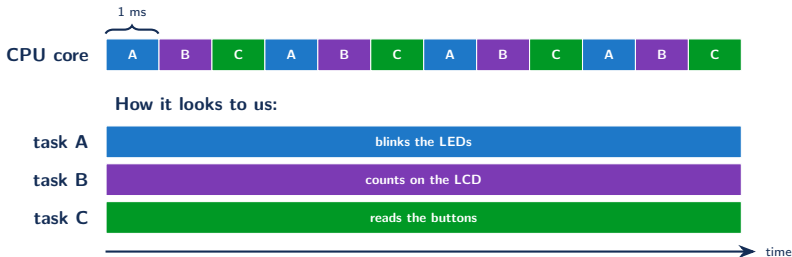
One processor, several tasks

In Task 2 each LED needed only a tiny piece of work, so it fit into the ISR. But real tasks are **long programs** with their own loops and waits: read a sensor, update the LCD, talk over Wi-Fi. They cannot run inside a short ISR.



How can three programs, each with its own endless loop, share **one core**?

The idea: take turns very quickly



Time slice

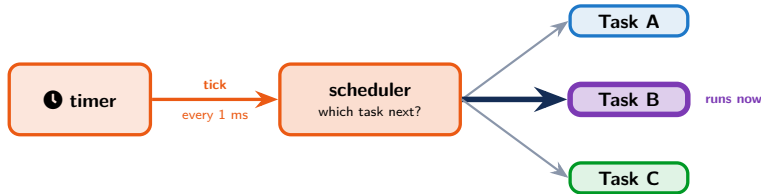
Each task runs for a short **time slice**, then the next one gets the core. The switches are so fast that all tasks seem to run at the same time.

Concurrency: several tasks make progress by **taking turns** on one core.

Several cores working at the same moment: *parallelism*. More on Tuesday.

Who decides when to switch?

The **scheduler** is the part of the system that decides **which task runs next** and switches to it.



Task 2
tick → **which LED?**

The scheduler
tick → **which task?**

The same idea as in Task 2: a regular **tick**, and a decision at every tick. On our ESP32 the tick comes every **1 ms**.

Simulator 4: time slices

Simulator task

Topic **Time slice**: tasks A, B and C take turns (round robin), one time slice = 1 ms. Task A starts at 0 ms. **Predict first**, then check.

1. Which task runs from **7 to 8 ms**?
2. How many task switches happen in **one second**?
3. Set the time slice to **500 ms**. Look at the row *how it looks to us*. Do the tasks still seem to run at the same time?

Solution: Simulator 4



Question 1

B: A, B, C, A, B, C, A, B.

Question 2

1000: one switch every 1 ms.

Question 3

No. Now you can see them take turns: the LED stops for a whole second while B and C run.

Concurrency works because the time slices are **short**: much shorter than anything we can notice.

What does a task need to continue later?

If we stop task A now and resume it later, **what must we remember?**

What does a task need to continue later?

If we stop task A now and resume it later, **what must we remember?**



PC

where the task stopped: the next
instruction

What does a task need to continue later?

If we stop task A now and resume it later, **what must we remember?**



PC

where the task stopped: the next instruction



Registers

the values it was working with, e.g. x5, x6 in Ripes

What does a task need to continue later?

If we stop task A now and resume it later, **what must we remember?**



PC

where the task stopped: the next instruction



Registers

the values it was working with, e.g. x5, x6 in Ripes



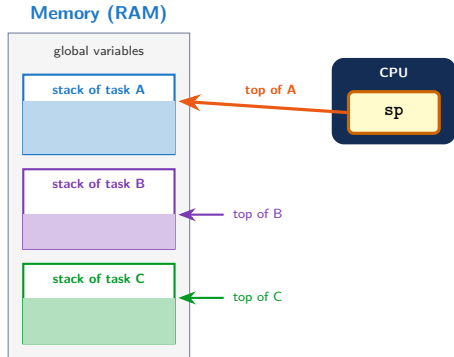
Stack

its local variables and the functions it is in

Together this is the task's **context**.

The ISR in Simulator 1 already saved t0: the same idea, but now we save **everything**.

Each task has its own stack



Its own stack

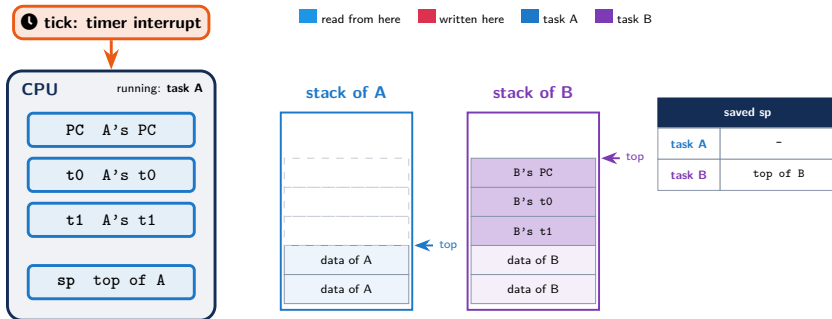
Each task has **its own stack** in memory: its local variables and saved registers.

The register `sp`

The stack pointer points to the stack of the task that is **running now**.

To switch tasks: save the registers on the old task's stack and **change** `sp`.

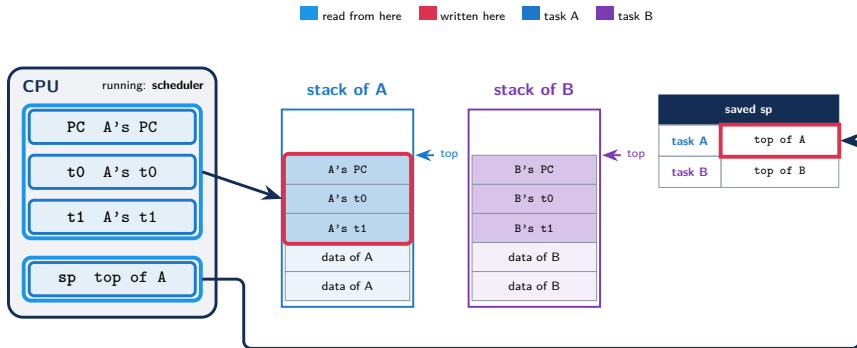
Context switch, step by step



1 Tick

Task A runs. The timer interrupt arrives and the CPU jumps to the scheduler.

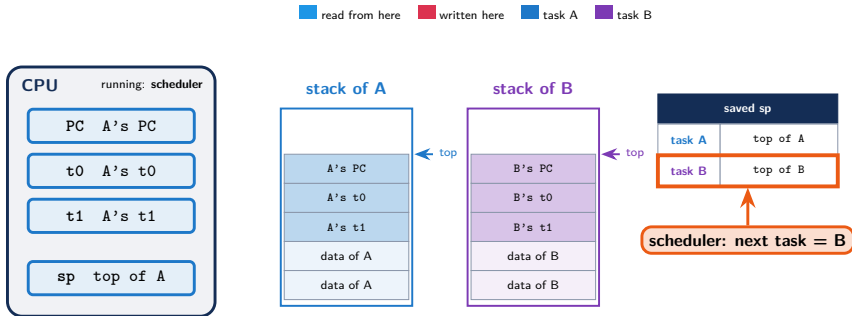
Context switch, step by step



2 Save A

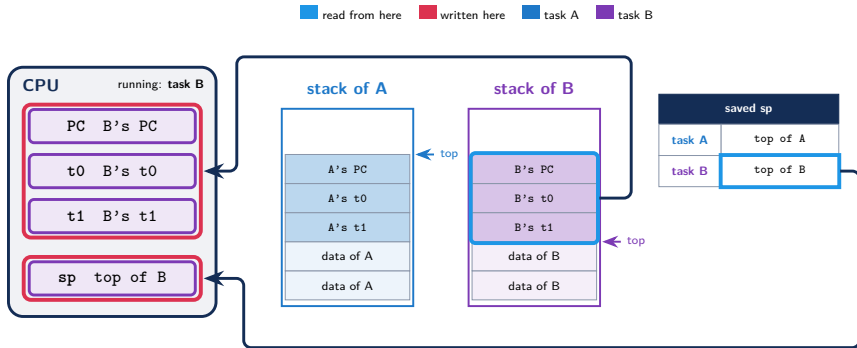
A's registers and PC go onto **stack A**. The new top of stack A goes into the table of saved stack pointers.

Context switch, step by step



- 3 **Choose B**
The scheduler picks the next task: **B**.

Context switch, step by step



4 Restore B

$sp \leftarrow$ the saved sp of B. B's registers and PC come back from stack B: B continues as if nothing happened. These 4 steps are a **context switch**.

Simulator 5: a context switch

Simulator task

Topic **Context switch**. Before each step, predict what will change. Then click **Next** and check.

At the start

	CPU (task A runs)	saved on stack B
PC	0x0210	0x0308
t0	7	42
t1	3	9
sp	0x2000	saved sp of B: 0x2FF4

Each register takes **4 bytes**. A context = PC, t0, t1.

1. Step 2, *Save A*: which values go onto stack A? What is the saved sp of A?
2. Step 4, *Restore B*: what are PC, t0, t1 and sp after the restore?
3. Where is A's t0 = 7 now? Who gives it back to A, and when?

Solution: Simulator 5

Question 1

0x0210, 7 and 3: 3 registers $\times$ 4 bytes = 12 = 0xC.

Saved sp of A: 0x2000 - 0xC = 0x1FF4.

Question 2

PC = 0x0308, t0 = 42, t1 = 9.

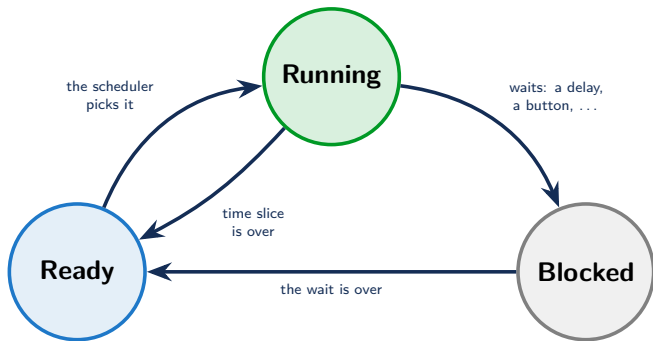
sp = 0x2FF4 + 0xC = 0x3000.

Question 3

On **stack A**. The scheduler restores it at the next switch back to A, exactly like B's values now.

Task A will never notice the break: its t0 waits for it on its own stack.

Task states



Running

On one core only **one** task runs at a time.

Ready

Could run, waits for its turn.

Blocked

Waits for something: a delay, a button. Gets **no** time slices.

Nobody is Ready?

Then the CPU runs a special task that does nothing: **IDLE**.

A Blocked task uses **no processor time**, unlike polling.

Which task runs next?

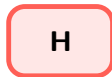
Round robin

Tasks with the **same priority** take turns, one time slice each.

Priorities

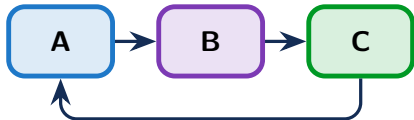
A Ready task with a **higher priority** always runs first. It even stops (preempts) a running task with a lower priority.

priority 2



when H is Ready, it runs first

priority 1



take turns: one time slice each

The scheduler picks the Ready task with the **highest priority**; equal priorities take turns.

Simulator 6: states, priorities and a shared counter

Simulator task

Topic **States and priorities**: A, B and C have priority 1. Task H has priority 2 and waits for the button.

1. At 2 ms, task C calls a 3 ms delay. In which state is C from 2 to 5 ms? Does it get any time slices?
2. Press the button: H becomes Ready. What happens to the task that is running?
3. What runs when A, B and C are all Blocked?

Then

Topic **Shared counter**: two tasks, each adds 1 to counter three times. The counter starts at 0.

4. **Predict** the final value.
5. Click **Run** three times and write down the results.

Solution: Simulator 6

Question 1

Blocked: no time slices. At 5 ms it becomes Ready again.

Question 2

It is stopped at once and goes to Ready. **H** runs. When H waits again, A, B and C continue.

Question 3

The **IDLE** task.

Questions 4 and 5

We expect **6**. But some runs give **4** or **5**, and the result changes from run to run.

Some additions were lost

Two tasks, six additions, and the counter did not count them all. Why? Keep this question: we will meet it again at the end of today.

A real scheduler: FreeRTOS

FreeRTOS is a small operating system for microcontrollers: mainly a **scheduler** plus tools that help tasks cooperate.

⚡ Free and open

Open source. Runs on many processors, including RISC-V.

🕒 RTOS

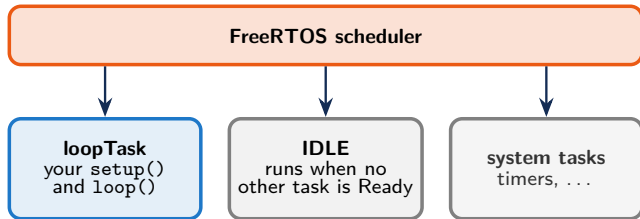
Real-time operating system: a task with a high priority reacts quickly.

📶 Everywhere

Sensors, home appliances, cars, ...

Everything from Part 2 is inside: the tick, time slices, states, priorities and the context switch.

You have used FreeRTOS all along



loopTask

On the ESP32, your `setup()` and `loop()` already run **inside a FreeRTOS task**.

`delay() = vTaskDelay()`

On the ESP32, `delay()` makes the task **Blocked**: while your `loop()` waited, it did not waste the CPU.

Nothing to install: we can create our own tasks.

Task 3: who runs loop()?



```
void setup() {
  Serial.begin(115200);
  Serial.print("Task: ");
  Serial.println(pcTaskGetName(NULL));
  Serial.print("Priority: ");
  Serial.println(uxTaskPriorityGet(NULL));
}

void loop() {
  delay(1000);
}
```

Task

Any Wokwi project: replace `setup()` and `loop()` with this code. Run it and open the **Serial Monitor**.

1. Which task runs `setup()`?
2. What is its priority?

NULL means: the task that is running now.

Solution: who runs `loop()`?

Serial Monitor



Task: `loopTask`
Priority: `1`

Questions 1 and 2

The task `loopTask`, with priority `1`.

Next

The tasks we create will also get priority `1`, so they take turns with `loopTask`.

Creating a task: xTaskCreate



```

void blinkTask(void *param) {
    for (;;) {                // forever
        digitalWrite(LED1, HIGH);
        vTaskDelay(pdMS_TO_TICKS(500));
        digitalWrite(LED1, LOW);
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

void setup() {
    pinMode(LED1, OUTPUT);
    xTaskCreate(blinkTask, "blink", 2048, NULL, 1, NULL);
}

```

The arguments of xTaskCreate

blinkTask	the task function: an endless loop
"blink"	a name, for debugging
2048	stack size, on the ESP32 in bytes
NULL	a parameter for the task
1	the priority
NULL	a handle (not needed here)

vTaskDelay makes the task **Blocked**: it uses no processor time while it waits.

Task 4: three LEDs as three tasks

Task

The same three LEDs as in Task 2, but now **without the timer**.

1. Remove the timer. Create **three tasks** with `xTaskCreate`: LED 1 changes every 300 ms, LED 2 every 500 ms, LED 3 every 700 ms.
2. Keep Loop: N on the LCD in `loop()`. Do the LEDs keep their rhythm?
3. In the task of LED 3, replace `vTaskDelay` with a **busy wait** (on the right). Do the other LEDs still blink? Now create this task with priority 2. What happens?

A busy wait for question 3



```
unsigned long t = millis();
while (millis() - t < 700) { }
```

Hint

One function can serve all three tasks: pass the pin and the period through `param`.

If you finish early: replace `vTaskDelay` with `vTaskDelayUntil`. What is the difference?

Solution: three LEDs as three tasks



```
struct Blink { int pin; int ms; };
Blink b1 = {4, 300}, b2 = {7, 500}, b3 = {3, 700};

void blinkTask(void *param) {
    Blink *b = (Blink *)param;
    pinMode(b->pin, OUTPUT);
    for (;;) {
        digitalWrite(b->pin, HIGH);
        vTaskDelay(pdMS_TO_TICKS(b->ms));
        digitalWrite(b->pin, LOW);
        vTaskDelay(pdMS_TO_TICKS(b->ms));
    }
}

// in setup():
xTaskCreate(blinkTask, "led1", 2048, &b1, 1, NULL);
xTaskCreate(blinkTask, "led2", 2048, &b2, 1, NULL);
xTaskCreate(blinkTask, "led3", 2048, &b3, 1, NULL);
```

Question 2

Yes. Each LED task is Blocked most of the time and gets the core within 1 ms when its delay is over.

Question 3, priority 1

The other LEDs still blink, but the busy task wastes its time slices: **polling again**.

Question 3, priority 2

The other LEDs and the LCD **stop**: the busy task is always Ready and has the highest priority.

If you finished early

`vTaskDelayUntil` wakes the task at fixed moments: small delays do not add up.

Two solutions of the same problem

	Task 2: one timer + ticks	Task 4: three FreeRTOS tasks
Who counts the ticks	you, in the ISR	the scheduler
The code of one LED	mixed into one ISR	its own simple loop
A fourth LED	change the ISR	one more <code>xTaskCreate</code>
Waiting	–	<code>vTaskDelay</code> : Blocked, no CPU
Cost	nothing extra	one stack per task (RAM)

Both use a timer tick. With FreeRTOS, the scheduler does the bookkeeping for you. Next: **look inside**, what did it do?

Look inside: a task in FreeRTOS

tasks.c (simplified)

```

    ● ● ●
typedef struct tskTaskControlBlock {
    volatile StackType_t *pxTopOfStack; /* MUST be first */
    ListItem_t    xStateListItem; /* Ready or Blocked list */
    UBaseType_t   uxPriority;      /* priority */
    StackType_t   *pxStack;      /* start of the stack */
    char          pcTaskName[16]; /* name */
    /* ... */
} tskTCB;
```

pxTopOfStack

The **saved sp** from our context switch. It is the **first** field.

xStateListItem

On which list the task is: **Ready** or **Blocked**.

uxPriority

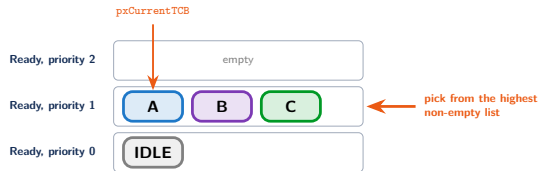
Its priority: 1 for our tasks.

TCB = Task Control Block: one per task. It is our table of saved stack pointers, and more.

Choosing the next task: vTaskSwitchContext

tasks.c (simplified)

```
void vTaskSwitchContext( void )
{
    /* ... */
    taskSELECT_HIGHEST_PRIORITY_TASK();
    /* now pxCurrentTCB points to
       the TCB of the next task */
}
```



Step 3 of the context switch

The scheduler takes the first task from the **highest non-empty** Ready list and stores its TCB in `pxCurrentTCB`. Tasks with the same priority take turns.

Saving and restoring registers (simplified)

Save the old task (step 2)

```

● ● ●
addi sp, sp, -CONTEXT_SIZE # room on its stack
sw   ra, 1*4(sp)           # save registers
sw   t0, 2*4(sp)
...
csrr t0, mepc              # where it stopped
sw   t0, 0(sp)             # save the PC
lw   t0, pxCurrentTCB      # its TCB
sw   sp, 0(t0)             # pxTopOfStack = sp

```

Restore the new task (step 4)

```

● ● ●
lw   t0, pxCurrentTCB      # the new TCB
lw   sp, 0(t0)             # sp = pxTopOfStack
lw   t0, 0(sp)             # its saved PC
csrw mepc, t0
lw   ra, 1*4(sp)           # restore registers
lw   t0, 2*4(sp)
...
addi sp, sp, CONTEXT_SIZE
mret                       # continue the task

```

Exactly our 4 steps from Part 2, in real code: lw and sw as in Ripes, and one TCB per task.

Simulator 7: the context switch with FreeRTOS names

Simulator task

Topic **Context switch** again, the same start as in Simulator 5. Turn on *FreeRTOS names*. **Before each step**, predict, then click **Next**.

1. Before step 2: to which TCB does `pxCurrentTCB` point?
2. After step 2: what is the value of `pxTopOfStack` in the TCB of A?
3. Step 3: what does `vTaskSwitchContext` change?
4. Step 4: which line of the assembly gives `sp` its new value? What is that value?

Solution: Simulator 7

Question 1

To the TCB of **A**: A was running.

Question 2

0x1FF4: the same saved sp as in Simulator 5.

Question 3

Only pxCurrentTCB: now it points to the TCB of **B**.

Question 4

lw sp, 0(t0), with t0 = the TCB of B: sp = 0x2FF4.
After the registers are restored, sp = 0x3000.

The table of saved stack pointers from Part 2 is the field pxTopOfStack in every TCB.

Task 5: a race between two tasks



```
const int N = 1000000;
volatile int counter = 0;
volatile bool doneA = false, doneB = false;

void incTask(void *param) {
    for (int i = 0; i < N; i++) {
        counter++;
    }
    *(volatile bool *)param = true;    // done
    vTaskDelete(NULL);                 // end this task
}
```

Task

Open the project **Race**. Two tasks increase **one** shared counter.

1. Start **two** incTask tasks with the same priority, with &doneA and &doneB as parameters.
2. In loop(), wait until both are done. Then show on the LCD: Goal: 2000000 and Got: N.
3. Run it several times. Is N always the same? Is it ever correct?

If you finish early: set N = 1000. Is the result correct now? Why?

Solution: a race between two tasks



```
void setup() {
    // the LCD as before
    xTaskCreate(incTask, "incA", 2048,
                (void *)&doneA, 1, NULL);
    xTaskCreate(incTask, "incB", 2048,
                (void *)&doneB, 1, NULL);
}

void loop() {
    if (doneA && doneB) {
        lcd.setCursor(0, 0);
        lcd.print("Goal: ");  lcd.print(2 * N);
        lcd.setCursor(0, 1);
        lcd.print("Got: ");   lcd.print(counter);
        while (true) delay(1000);
    }
    delay(10);           // wait without wasting the CPU
}
```

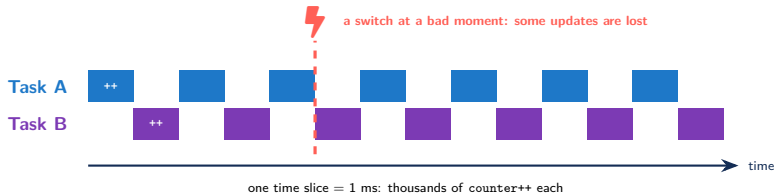
Question 3

N is **smaller** than 2 000 000 and **different** every time. It is never correct.

If you finished early

With N = 1000 each task finishes within one time slice: no switch comes in the middle, so the result is correct.

What happened?



This is a **race condition**: the result depends on **when** the scheduler switches the tasks.

We saw it twice

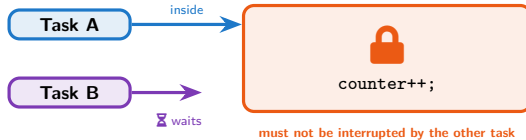
Simulator 6: six additions, but the counter showed 4 or 5. Task 5: two million additions, but the counter got less.

The question

Why? `counter++` is **one line**. How can a switch fall *in the middle* of it?

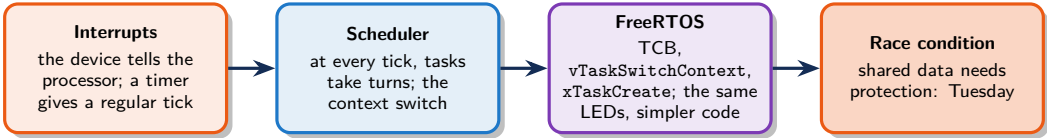
How do we fix this?

We need a way to say: *while I update the counter, nobody else may touch it.*



Tomorrow: **why** updates are lost, and how to fix it with **threads, mutexes and atomic operations**, in C++ on your laptops.

What we saw today



A brief history of concurrency: it all started with a button.

Check yourself

1. Why does an interrupt not miss a short press, while polling can?
2. What does the CPU save when an interrupt arrives, and why does the main program not notice the break?
3. What is a context switch? What is saved, and where?
4. Why does a task that waits with `vTaskDelay` not waste CPU time, while a busy loop does?

Try to answer each question in **2–3 sentences**.

Before Tuesday

From Tuesday we write **C++** on your own laptops. Install a compiler with **C++20** support (g++, clang or MSVC). Instructions will be sent today.

hello.cpp: check your installation

```
● ● ●
#include <iostream>
#include <thread>

int main() {
    std::thread t([] {
        std::cout << "Hello from a thread\n";
    });
    t.join();
}
```

Compile and run

```
● ● ●
g++ -std=c++20 -pthread hello.cpp -o hello
./hello
```

If it prints Hello from a thread, you are ready for Tuesday.